

swtpcsim — Changelog

2026-09-27 · 8ae2a01

A simulation of the SWTPC 6800 on the SS-50/SS-30 bus, and the MITS Altair 680b

Changelog

Every release of **swtpcsim**, newest first. Each entry is the short version — what you can do here that you could not do in the release before it. The User Manual describes the program as it is now; this document is the record of how it got there.

0.0.1

The first release — a simulator of **Motorola 6800 and 6809** machines (the SWTPC 6800 and 6809 on the SS-50/SS-30 bus, and the MITS Altair 680b), in C++20, with **nothing to fetch**: the TOML parser, the JSON encoder and the line editor are all in-tree, so a fresh clone builds with a C++20 compiler and CMake and no network. SDL3 is optional and detected, never required.

Machines that boot from a bare command line

- **swtpcsim swtpc** brings up an SWTPC 6800 under **SWTBUG**, its ROM monitor, with an MP-S serial console and a DC-4 floppy controller. **FLEX 2.0** boots from drive 0.
- **swtpcsim swtpc09** brings up an SWTPC 6809: the **MP-09** processor board under **S-BUG 1.8**, with 56K of RAM, an MP-S console at **E004** and a DC-4 floppy controller. **FLEX9 2.8:3** boots from `examples/flex9/` with S-BUG's **U** command.
- **swtpcsim altair680** brings up a MITS **Altair 680b** under **MON680**, the 680b's own ROM monitor, on a single 6850 ACIA — and the **KCACR** Kansas City audio-cassette interface loads and saves off tape.
- **swtpcsim cp68** boots **TSC CP/68** from a DC-4 disk.

The boards behind them

A Motorola **6800** CPU; a **memory** board that carries RAM, scratchpad RAM and the monitor ROM as a list of regions; the SWTPC **MP-S** serial console; the Altair 680b **onboard** and **Universal I/O** serial/parallel boards; the **KCACR** cassette; and the SWTPC **DC-4** WD179x floppy controller.

A Motorola **6809** CPU board (**6809**) as well. No built-in machine uses it yet, but a machine file can: **DISASM** and **EDIT** speak 6809, with every indexed mode, and **SHOW BUS IRQ** shows its seven vectors.

The SWTPC **MP-09** 6809 processor board (**mp09**), with its DAT address translator and the **S-BUG 1.8** monitor (**builtin:sbug**) in its ROM socket. S-BUG signs on (**S-BUG 1.8 - 56K**) over an MP-S console at **E004** . The MP-S and DC-4 **base** now also accepts the **E000 – E01C** I/O window that a 6809 system's motherboard uses.

The SWTPC **MP-T interrupt timer** (`mpt`): a 6820 PIA and an MK5009 time base in an SS-30 slot, interrupting the 6800 at any of its intervals from 1 μ s to 1 hour, with its other side as an input port (unit `in`). `examples/mpt/` runs SWTPC's **INTCLK** clock program on it — corrected from the published copy, whose interrupt vector was swapped — and ticks once a second.

The SWTPC **MP-ID interface driver** (`mpid`), in the `swtpc09` machine at `E080` : a **6840** timer counting the 50/60 Hz power line, and a PIA with a parallel printer port (unit `lpt`). It is FLEX9's timer, so FLEX9 no longer says `Timer not available.` at boot, and `TIME` reports how long a command took.

The monitor prompt and the debugger

Drive a running machine from the `swtpcsim>` prompt: fit and configure boards (`BOARDS` , `SHOW BOARD` , `SET`), mount disks and tapes, wire serial lines (`CONNECT`), and script a boot with `STARTUP` . The debugger reads the 6800: `DISASM` and the byte-at-a-time `EDIT` assembler speak 6800, `BREAK` / `HISTORY` / watchpoints and conditional breaks (`BREAK ... IF`) stop and trace it, `SYMBOLS` loads a Motorola **as0/as9** listing so disassembly reads by name, and `LOAD` reads a Motorola **S-record** (`.S19`) as well as Intel HEX. `SNAPSHOT` / `RESTORE` save and reload a whole machine.

Driving a machine over MCP

`swtpcsim <machine> --mcp` exposes the machine to an AI assistant as line-delimited JSON-RPC, with typed tools for running, stepping, breakpoints, memory and disassembly rather than a text prompt to screen-scrape. `--mirror` opens a live socket onto the same console, so a person can watch and take over the keyboard. A `run` that will not end can be stopped with a `notifications/cancelled` or a `^C`, and a `status` tool answers at once even while a `run` is in progress. Every tool checks its arguments, so `"from": "0xE0D0"` is refused with the number to send instead of running from 0.

The windows

With SDL3 present, a serial line can open a **built-in terminal window** the simulator draws itself (`CONNECT sio0:a terminal`), and video boards each open their own host window. A headless build refuses the endpoint cleanly rather than opening a line nobody can see.

The package, and the documents in it

Each platform archive holds the program, the **User Manual**, the **monitor** and **debugger** references, this changelog, a cheatsheet, both licences, and `examples/` — `flex` , `cp68` , `altair680` , `mpt` and a `debugger` walkthrough, media included. SDL3 is linked **statically**, so there is nothing to install beside the binary. Unzip it and run it; nothing needs fetching first.